

Exercise 8: Multi-Layer Perceptrons - Backpropagation

Task 1:

Answer the following questions:

- Why can't we just use the delta rule for learning weights of hidden layers?
- How far is training neural networks non-deterministic? How does randomness influence training speed and/or resulting model performance?

Task 2: Batch Gradient Descent

We are looking at a classic Multi-Layer Perceptron (MLP) architecture with 2 input neurons, 2 hidden neurons, and 2 output neurons.

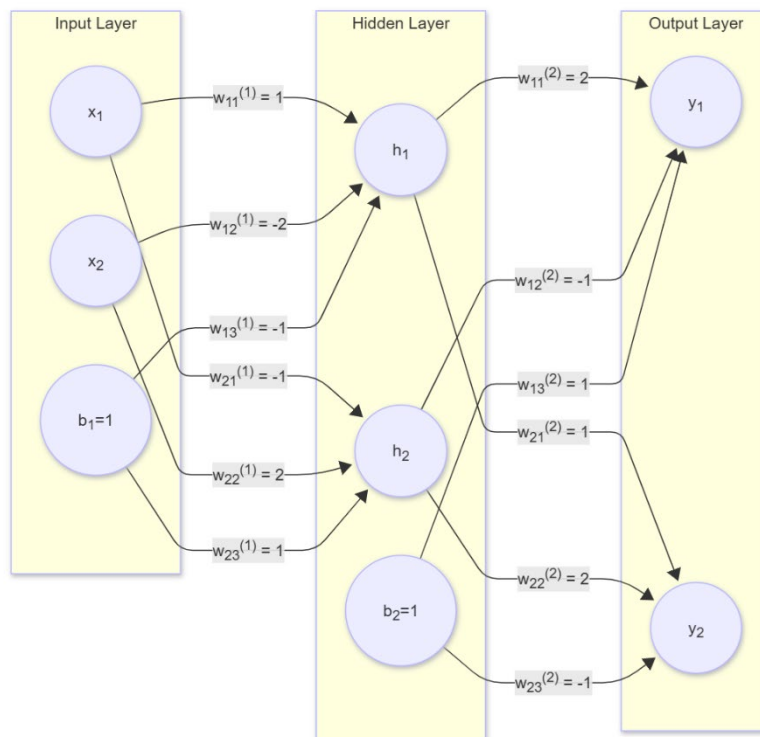
- Activation Function:** The standard Sigmoid function $\sigma(x) = \frac{1}{1+e^{-x}}$ is used for all neurons.
- Loss Function:** $E = \frac{1}{2} \sum (o_i - y_i)^2$. The sum of Squared Errors (SSE).

Since the quadratic loss is used and all layers use the exact same activation function, you can use the formula for backpropagation from the lecture:

$$\delta_i^{(l)} = \begin{cases} f'(net_i)(o_i - y_i) & \text{if output layer} \\ f'(net_i) \sum_k (\delta_k^{(l+1)} w_{ki}^{(l+1)}) & \text{else} \end{cases}$$

Where o_i is the desired output and y_i is the actual output.

Below is the structure of the network:



Use the exact and rounded values (rounded to 2 decimal places) from the Sigmoid lookup table to solve at least the Forward Pass without a calculator.

(Note: Please also round all intermediate steps to 2 decimal places.)

You are given a batch of three training samples:

1. $S^{(1)} : X = [1, 1]$, Target $Y = [1, 0]$
2. $S^{(2)} : X = [3, 1]$, Target $Y = [0, 1]$
3. $S^{(3)} : X = [0, 0]$, Target $Y = [0.5, 0.5]$

Calculate the forward pass for all three samples. **Determine** the hidden activations (h_1, h_2) and the final outputs (y_1, y_2) for each sample. Use the lookup table.

Now we want to update the weights. But how to do that? Before we can update any weights, we need to determine the local errors for each training sample individually. Use the generic formula provided from the lecture to calculate the following values for each of the three samples ($S^{(1)}, S^{(2)}, S^{(3)}$):

1. The error components of the output layer: $\delta_1^{(2)}$ and $\delta_2^{(2)}$
2. The error components of the hidden layer: $\delta_1^{(1)}$ and $\delta_2^{(1)}$

The mathematical change of a weight $w_{ij}^{(l)}$ caused by a single, isolated data sample (s) is defined using our lecture's delta rule format:

$$\Delta w_{ij}^{(l)} = \eta \cdot \delta_i^{(l)} \cdot o_j^{(l-1)}$$

Where η is the learning rate and $o_j^{(l-1)}$ the output of the j neuron of the previous layer.

In **Batch Gradient Descent**, however, we follow a different protocol: We evaluate the *entire* batch (all 3 samples), calculate all individual updates while keeping the weights strictly frozen, and perform a single, collective update at the very end. The total weight change for the whole batch is defined as the mean (average) of the individual weight changes across all processed samples:

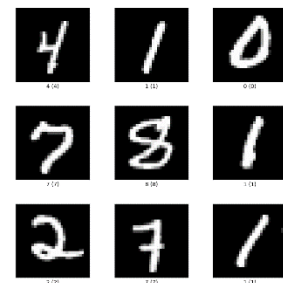
$$\Delta w_{ij}^{(l, \text{batch})} = \frac{1}{3} \sum_{s=1}^3 \Delta w_{ij}^{(l, s)}$$

1. Compute the final batch weight adjustments for the output layer $\Delta w_{ij}^{(2, \text{batch})}$ and for the hidden layer $\Delta w_{ij}^{(1, \text{batch})}$ with a learning rate of 1.
2. Briefly explain in your own words why machine learning frameworks average the gradients over a batch instead of correcting the weights immediately after looking at a single data point. What is the fundamental benefit for the training process?

Task 3:

Use the notebook *mnist.ipynb* provided in the Moodle course as a baseline to implement an MLP for solving the MNIST digits dataset (<https://www.tensorflow.org/datasets/catalog/mnist>).

Each sample has a 28x28 grayscale image of a single handwritten digit (0-9), as shown on the right, and the corresponding expected output, again 0-9.



Sigmoid Lookup Table

x	$\sigma(x)$	$\sigma'(x)$		x	$\sigma(x)$	$\sigma'(x)$		x	$\sigma(x)$	$\sigma'(x)$		x	$\sigma(x)$	$\sigma'(x)$
-2.02	0.12	0.11		-0.02	0.50	0.25		0.34	0.58	0.24		0.73	0.67	0.22
-2.01	0.12	0.11		-0.01	0.50	0.25		0.35	0.59	0.24		0.74	0.68	0.22
-2.00	0.12	0.11		0.00	0.50	0.25		0.36	0.59	0.24		...	...	...
-1.99	0.12	0.11		0.01	0.50	0.25		0.37	0.59	0.24		0.79	0.69	0.21
-1.98	0.12	0.11		0.02	0.50	0.25		0.38	0.59	0.24		0.80	0.69	0.21
...	...	...		...	...	...		...	...	...		0.81	0.69	0.21
-1.02	0.26	0.19		0.10	0.52	0.25		0.48	0.62	0.24		0.82	0.70	0.21
-1.01	0.27	0.20		0.11	0.53	0.25		0.49	0.62	0.24		0.83	0.70	0.21
-1.00	0.27	0.20		0.12	0.53	0.25		0.50	0.62	0.24		...	...	...
-0.99	0.27	0.20		...	...	...		0.51	0.63	0.23		0.86	0.70	0.21
-0.97	0.27	0.20		0.18	0.54	0.25		0.52	0.63	0.23		0.87	0.70	0.21
...	...	...		0.19	0.55	0.25		...	...	...		0.88	0.71	0.21
				0.20	0.55	0.25		0.57	0.64	0.23		0.89	0.71	0.21
				0.21	0.55	0.25		0.58	0.64	0.23		0.90	0.71	0.21
				0.22	0.55	0.25		0.59	0.64	0.23		...	...	...
				0.23	0.56	0.25		0.60	0.65	0.23		1.48	0.81	0.15
				0.24	0.56	0.25		...	...	...		1.49	0.82	0.15
				0.25	0.56	0.25		0.65	0.66	0.22		1.50	0.82	0.15
				0.26	0.56	0.25		0.66	0.66	0.22		1.51	0.82	0.15
				...	...	...		0.67	0.66	0.22		1.52	0.82	0.15
								0.68	0.66	0.22		...	...	...
								...	...	...		1.98	0.88	0.11
								0.71	0.67	0.22		1.99	0.88	0.11
								0.72	0.67	0.22		2.00	0.88	0.11
												2.01	0.88	0.11
												2.02	0.88	0.11